BPHIST

Website Design & Implementation Plan

A practical blueprint for a modern, SEO-friendly college website with staff-managed content.

Recommended architecture
Next.js · TypeScript · Tailwind CSS · Payload CMS · PostgreSQL

Scope: public website, news, editable content, media, permissions, deployment and SEO.

Planning estimate: approximately 4–6 weeks for a first release, assuming one developer, timely content approvals and no major integrations.

Contents

- 1. Executive recommendation
- 2. Goals and scope
- 3. Technology stack
- 4. Application architecture
- 5. Public website design
- 6. CMS content model
- 7. Permissions and publishing workflow
- 8. Deployment and operations
- 9. SEO and discoverability
- 10. Implementation phases
- 11. Suggested project structure
- 12. Security and launch checklist
- 13. Risks and next actions

Decision summary

Area	Recommendation
Application	Next.js App Router + TypeScript
UI	Tailwind CSS and reusable accessible components
CMS	Payload CMS integrated into the Next.js application
Database	PostgreSQL
Media	Persistent object storage and resized image variants
Hosting	Managed Next.js-compatible hosting + managed database
First release	Public pages, programmes/admissions, news, media, SEO and roles

1. Executive recommendation

Build BPHIST as a custom Next.js website with Payload CMS integrated into the same application. Staff should be able to change approved text, images, news and admission information through a protected admin panel, without editing source code.

Separate **content management** from **website design**: staff maintain approved content; the developer maintains layout, reusable components, schemas, integrations and technical configuration.

Do not build a CMS from scratch or split the first version into several independent services without a clear need. WordPress is a valid alternative if the overriding priority is the easiest possible editing experience for nontechnical staff; this plan favours Next.js + Payload for a developer-led custom build.

2. Goals and scope

- Present BPHIST clearly on mobile, tablet and desktop.
- Explain the college, programmes, admissions, facilities and contact details.
- Let authorised staff publish news and update content without a code deployment.
- Manage photos and documents through a media library.
- Include SEO, permissions, backups and a maintainable deployment process.

Included in first release

- Home, About, Programmes, Admissions, News, Campus/Gallery and Contact pages.
- Editable page sections, programme/admission content, news/events and college settings.
- Staff accounts, role permissions, drafts, preview and publishing.
- SEO metadata, sitemap, robots rules, social-sharing metadata, deployment, backups and staff training.

Defer until later

Student portal, online application processing, payments, live chat, complex analytics dashboards, elaborate animations and unrestricted drag-and-drop page building. These add security and maintenance obligations beyond the current scope.

3. Technology stack

Layer	Choice	Responsibility
Frontend/application	Next.js App Router + TypeScript	Public pages, routes, rendering, server logic and metadata.
Styling	Tailwind CSS	Responsive layouts, design tokens and consistent components.
CMS/admin	Payload CMS	Content collections, staff UI, authentication, publishing and access control.
Database	PostgreSQL	Pages, news, programmes, users and structured settings.
Media	Persistent object storage	Uploaded images, resized variants and downloadable documents.
Email (if needed)	Transactional email provider	Password resets and optional editorial notifications.
DNS/edge	Domain DNS; optional Cloudflare	DNS, caching and suitable security features.
Monitoring	Host logs and uptime/error monitoring	Detect outages, deployment failures and application errors.

Choose compatible stable versions at implementation time, record them in the lockfile and avoid unnecessary major upgrades during launch.

4. Application architecture

Public visitor → Next.js website → published CMS content

Staff editor → protected /admin → Payload access rules → PostgreSQL and media storage

Keep the public rendering layer, CMS configuration, collections and reusable components logically separated. Public requests must never expose draft or privileged administrative data.

Component	Responsibility
Public website	Renders published pages and articles; provides navigation, accessible UI and SEO metadata.
Payload admin	Authenticated interface for editors and administrators.
Collections	Define fields, validation, relationships and publishing state.
PostgreSQL	Persistent structured content and user records.
Object storage	Persistent uploads; avoid temporary application filesystems.
Deployment pipeline	Build, test and deploy reviewed code; content edits should not require redeployment.

Caching and publishing

Use static generation or server rendering where appropriate. When content is published, invalidate or revalidate affected pages/tags so updates appear promptly. Test this behaviour explicitly; do not assume a CMS save automatically refreshes every cached page.

5. Public website design and navigation

Visual direction

- Use BPHIST blue as the primary brand colour, white backgrounds and restrained light-blue accents.
- Use the official logo without altering its proportions or details.
- Use authentic, approved campus, classroom, laboratory and student-life photographs.
- Prioritise mobile-first layouts, readable type, strong contrast, keyboard accessibility and fast images.
- Keep the hero focused on one headline and a primary action; avoid a carousel that hides the main message.

Recommended navigation

Page	Purpose
Mwanzo / Home	Introduction, admission CTA, featured programmes, facilities and latest news.
Kuhusu Sisi / About	College profile, mission, vision, values and verified registration information.
Kozi / Programmes	Programme listings and pages with duration and entry requirements.
Udahili / Admissions	Current intake, steps, requirements, deadlines, application links and guide.
Habari na Matukio / News	Announcements, activities, events and notices.
Maisha Chuoni / Campus	Facilities, laboratories, classrooms and campus gallery.
Mawasiliano / Contact	Address, phone, email, social links and directions.

Homepage section order

- Compact contact/social strip (optional).
- Header, logo, navigation and admission CTA.
- Hero headline, short introduction, authentic image and primary/secondary actions.
- Featured programmes and course facts.
- Admission steps and key requirements.
- Why BPHIST / facilities and practical learning.
- Campus/gallery highlights.
- Latest news and events.
- Testimonials only when genuine and approved.
- FAQ, contact section, location and footer.

Design rule

Build fixed reusable sections first. Staff may edit approved text, media, links, visibility and ordering where supported, but should not accidentally alter the global layout or design system.

6. CMS content model

Collection / group	Recommended fields and behaviour
Pages	Title, slug, page type, content sections, status, SEO title/description, social image, updated date.
News & Events	Title, slug, summary, body, cover image, alt text, category, author, publication date, status, links.
Programmes	Name, slug, overview, duration, award/level, entry requirements, image and status.
Admissions / Intake	Academic year, dates, deadline, requirements, application instructions/link, guide, contact and status.
Media	File, descriptive name, alt text/caption, crop/focal point if supported, generated sizes and usage notes.
FAQs	Question, answer, category, order and visibility.
Global Settings	Approved logo, contacts, address, social links, footer and default SEO settings.
Users	Named accounts, role, active status and least-privilege access.

Reusable page sections

Use a controlled set of components: Hero, Rich Text, Programme Cards, News Feed, Image Gallery, CTA Banner, FAQ List and Contact Details. Each has a defined schema and validation. Do not allow arbitrary executable markup from editors.

Image replacement workflow

- Editor signs in to /admin and opens the relevant page/section.
- Editor selects an existing media item or uploads an approved file.
- Editor adds meaningful alt text where the image conveys information.
- Editor previews and saves a draft or publishes according to permissions.
- The site refreshes affected pages and serves the image from persistent storage.

News workflow

Create article → add summary and body → select cover image/category → preview → approval if required → publish → update or archive. Keep article URLs stable wherever possible.

7. Permissions and publishing workflow

Role	Responsibilities	Restrictions
Administrator	Users/roles, all content, settings, media and publishing.	Trusted system owner only; protect strongly.
Content editor	News, events, page content and media; submit drafts.	No user management or technical configuration.
Admissions editor	Programme/admission content and related downloads.	No user management or unrelated settings.
Approver/publisher	Review drafts and publish assigned content areas.	No technical settings unless separately authorised.
Developer	Source code, schemas, integrations, deployment and technical settings.	Limit and account for production access.

Workflow

Draft → Preview → Approval (if required) → Published → Updated or Archived

For a small college, one person may initially be both editor and approver. Keep roles separable as the team grows.

Account and permission requirements

- Give each staff member an individual account; never share one college login.
- Enforce access on the server/CMS, not only by hiding interface controls.
- Restrict who can change global settings, roles, integrations and production configuration.
- Use strong unique passwords and MFA if supported by the chosen setup.
- Review accounts when staff change roles or leave.
- Record significant admission-content changes and who made them.

8. Deployment and production operations

Area	Plan
Domain/DNS	Confirm domain ownership and registrar/DNS access; configure HTTPS and correct DNS records.
Application	Managed Next.js-compatible host or managed Node.js environment; verify Payload and image-processing compatibility.
Database	Managed PostgreSQL with backups, access controls and a restore procedure.
Media	Persistent object storage for uploads, originals, resized images and PDFs.
Email	Transactional provider for password resets/notifications; authenticate sending domain.
Environments	Separate local development, staging and production; never commit secrets.
Backups	Back up database and media independently; test restoration.
Monitoring	Review logs, deployment errors, uptime and background-task failures if introduced.

Before choosing a provider

Compare current pricing, bandwidth, database storage, backup retention, media egress, regional availability, support and plan limits. Do not assume a free tier is suitable for permanent production use.

9. SEO and discoverability

Area	Implementation
Metadata	Editable title and description per important page/article; sensible global defaults.
URLs	Readable, stable slugs, e.g. /kozi/nama-ya-programme and /habari/jina-la-tangazo.
Semantic structure	Meaningful HTML, logical heading hierarchy, descriptive links and accessible controls.
Images	Responsive optimised sizes, suitable formats and descriptive alt text for informative images.
Technical files	Generated sitemap.xml, correct robots.txt and canonical URLs where needed.
Social previews	Open Graph-compatible metadata and appropriate share image.
Structured data	Accurate, applicable data for institution, courses and news; validate before launch.
Internal links	Connect programme pages, admissions, relevant news and contact details.
Monitoring	Verify the domain in Google Search Console; monitor indexing, queries and errors.
Content quality	Publish useful original content and verify programme details/deadlines with responsible staff.

SEO cautions

SEO-friendly code does not guarantee rankings, and structured data does not guarantee rich search results. Avoid copied filler, duplicate pages, misleading claims and outdated intake dates. Assign a staff owner to review admissions content regularly.

10. Implementation phases

Phase	Estimate	Work	Deliverable
1. Requirements	2–3 days	Sitemap, branding, copy, media, roles and approvals.	Approved sitemap and content checklist.
2. Foundation/CMS	4–6 days	Project, database, auth, collections and access rules.	Working admin and initial schemas.
3. Public UI	5–7 days	Shared layout, responsive templates and reusable sections.	Public site with sample content.
4. CMS integration	4–6 days	Pages/news/media, preview, publishing and SEO.	Staff update content without code changes.
5. QA and launch	4–6 days	Permissions, responsive UI, SEO, backups and deployment.	Live site, staff training and runbook.

Total planning range: approximately 4–6 weeks for one developer, assuming prompt content approvals and no major integrations. Content preparation, image selection, domain access or hosting constraints may extend the schedule.

Phase gates

- Agree the content model and roles before building every page.
- Have editors test real workflows before launch.
- Do not declare backup readiness until a restore test succeeds.
- Do not publish unverified programme or admission requirements.

11. Suggested project structure

```
bphist-website/
■■■ src/
■ ■■■ app/
■ ■ ■■■ (website)/
■ ■ ■ ■■■ page.tsx
■ ■ ■ ■■■ kuhusu-sisi/
■ ■ ■ ■■■ kozi/
■ ■ ■ ■■■ udahili/
■ ■ ■ ■■■ habari/
■ ■ ■ ■■■ mawasiliano/
■ ■ ■■■ (payload)/
■ ■■■ components/
■ ■ ■■■ layout/
■ ■ ■■■ home/
■ ■ ■■■ news/
■ ■ ■■■ shared/
■ ■■■ collections/
■ ■■■ Users.ts
■ ■■■ Pages.ts
■ ■■■ News.ts
■ ■■■ Programmes.ts
■ ■■■ Media.ts
■■■ payload.config.ts
■■■ public/
■■■ tests/
■■■ README.md
```

This is illustrative rather than rigid. Follow the selected Payload template/version conventions and keep CMS schemas separate from visual components.

12. Security, QA and acceptance

Security baseline

- Store secrets in environment variables or a secret manager; never commit them to Git.
- Enforce HTTPS and least-privilege access to the admin panel.
- Validate upload types and file size; restrict administrative operations.
- Keep dependencies patched and review security advisories.
- Separate development, staging and production.
- Back up database and media; restrict backup access and test recovery.
- Do not publish personal or sensitive student information without a legitimate, approved reason.

Launch acceptance checklist

- ■ Staff can log in without developer assistance.
- ■ Editor can publish news and upload a cover image.
- ■ Staff can replace a homepage image without code changes.
- ■ Draft content stays hidden from public visitors.
- ■ Admissions staff can update approved programme/intake details.
- ■ Unauthorised users cannot manage accounts or sensitive settings.
- ■ Site works on mobile and desktop.
- ■ Titles, descriptions, sitemap and robots rules are correct.
- ■ Database and media backups can be restored.
- ■ Staff have a short editing and publishing guide.

13. Risks and next actions

Risk / decision	Action
Content not ready	Assign owners for official copy, programme details, dates, contacts and photos.
Unclear domain ownership	Confirm registrar, DNS and renewal access/responsibility.
Hosting cost or limits	Compare total app, database, media, bandwidth, backup and email cost.
Staff publish errors	Use preview, validation, permissions and approval for important content.
Stale admission details	Assign an admissions owner and review dates; archive expired notices.
Vendor dependency	Keep code in college-controlled Git and document setup/export/backup steps.
Scope creep	Plan applications, student accounts and payments as separate projects.

Recommended first actions

- Confirm domain and hosting account ownership.
- Appoint one college content owner and one technical owner.
- Collect approved logo, campus photos, contacts and programme/admission information.
- Approve the sitemap and homepage wireframe.
- Create the repository and initialise Next.js + Payload.
- Build Users, Media, Pages, News and Programmes collections before populating every page.

Final recommendation

Deliver a focused first release with Next.js + TypeScript + Tailwind CSS + Payload CMS + PostgreSQL, managed hosting and persistent media storage. Keep the CMS simple, enforce server-side permissions, and keep visual design and technical configuration under developer control. Launch the essentials well, then expand only when the college has a clear need and an owner for the added maintenance.

Technical references to consult during implementation

Use the official Next.js documentation for metadata, robots and sitemap generation; Payload documentation for admin, access control and media uploads; and Google Search Central guidance for SEO and structured data. Verify current documentation and version compatibility when implementation starts.